

Buenas Prácticas

Ingeniería de Sistemas

Universidad Austral — 2026

Un proyecto de software no se rompe de golpe

El síntoma

"¿Por qué este cambio de una línea tarda dos semanas?"

Nadie decidió que fuera así.

Cada paso individual fue razonable. El resultado, no.

Entropía

*Segunda ley de la termodinámica:
el desorden de un sistema cerrado no puede reducirse.
Sólo puede permanecer igual o **aumentar**.*

Una medida de ese desorden es la **entropía**.

Aplicado a software

*Cuando un sistema es modificado, su desorden — su entropía —
tiende a aumentar.*

Y un sistema de software se modifica **todo el tiempo**. Ése es su punto.

La teoría de las ventanas rotas



Qué dice la teoría

Un edificio con **una** ventana rota y sin reparar
termina con **todas** las ventanas rotas.

La señal que transmite no es "acá falta un vidrio".
Es "**acá a nadie le importa**".

*James Q. Wilson y George L. Kelling — "Broken Windows", **The Atlantic**, 1982*

En código

El primer `TODO` que nadie resuelve.

El primer test comentado "por ahora".

La primera regla del *linter* desactivada para salir del paso.

Cada uno **baja el costo** de hacer el siguiente.

*La aplicación al software es de Hunt y Thomas — **The Pragmatic Programmer**, 1999*

Ninguna práctica sobrevive sola

Una convención que **nadie hace cumplir** se pudre.

Un test que **nadie corre** se pudre.

Una documentación que **nadie actualiza** miente.

Lo que las sostiene es sacarlas de la **voluntad individual**:

Automatización — se acuerda la máquina

Proceso — se acuerda el equipo

El mapa de hoy

1. Acordar cómo se escribe — **convenciones**
2. Verificar sin ejecutar — **análisis estático**
3. Verificar ejecutando — **tests**
4. Construir siempre igual — **build**
5. Registrar el cambio — **versionado**
6. Revisar entre humanos — **code reviews**
7. Transmitir el conocimiento — **documentación**
8. Construir tus propias **herramientas**

1 • Acordar cómo se escribe

Convenciones

¿TABS o espacios?



Hasta hay una escena de [Silicon Valley](#) sobre esto.

Una convención abarca más de lo que parece

- **Formateo** — indentación, llaves, largo de línea
- **Estructura de directorios** — dónde va cada cosa
- **Nombres** — archivos, tipos, variables, tests
- **Idioms del lenguaje** — qué construcciones se usan y cuáles no

¿Cuál elegimos?

No hay respuesta correcta — hay respuesta **acordada**.

El valor no está en cuál, sino en que el código sea **predecible**:

- La atención va al **problema**, no a descifrar el estilo
- Los *diffs* muestran el **cambio real**, no reformateos
- Quien entra al equipo **produce antes**

Y casi nunca hay que elegir: **la industria ya eligió**.

¿Qué cambió acá?

PR #3 — proyecto **sin** formato estándar

1 file changed, 77 insertions(+), 79 deletions(-)

PR #2 — el **mismo** cambio, con formato estándar

1 file changed, 1 insertion(+), 1 deletion(-)

Y ya están escritas

- [Google Style Guides](#) — para casi todos los lenguajes
- [awesome-guidelines](#) — índice de guías por lenguaje

La mayoría de los lenguajes modernos ya traen un formateo canónico y un *formatter* oficial.

Y después, dejar de discutirlo

La convención se **delega en una herramienta**.

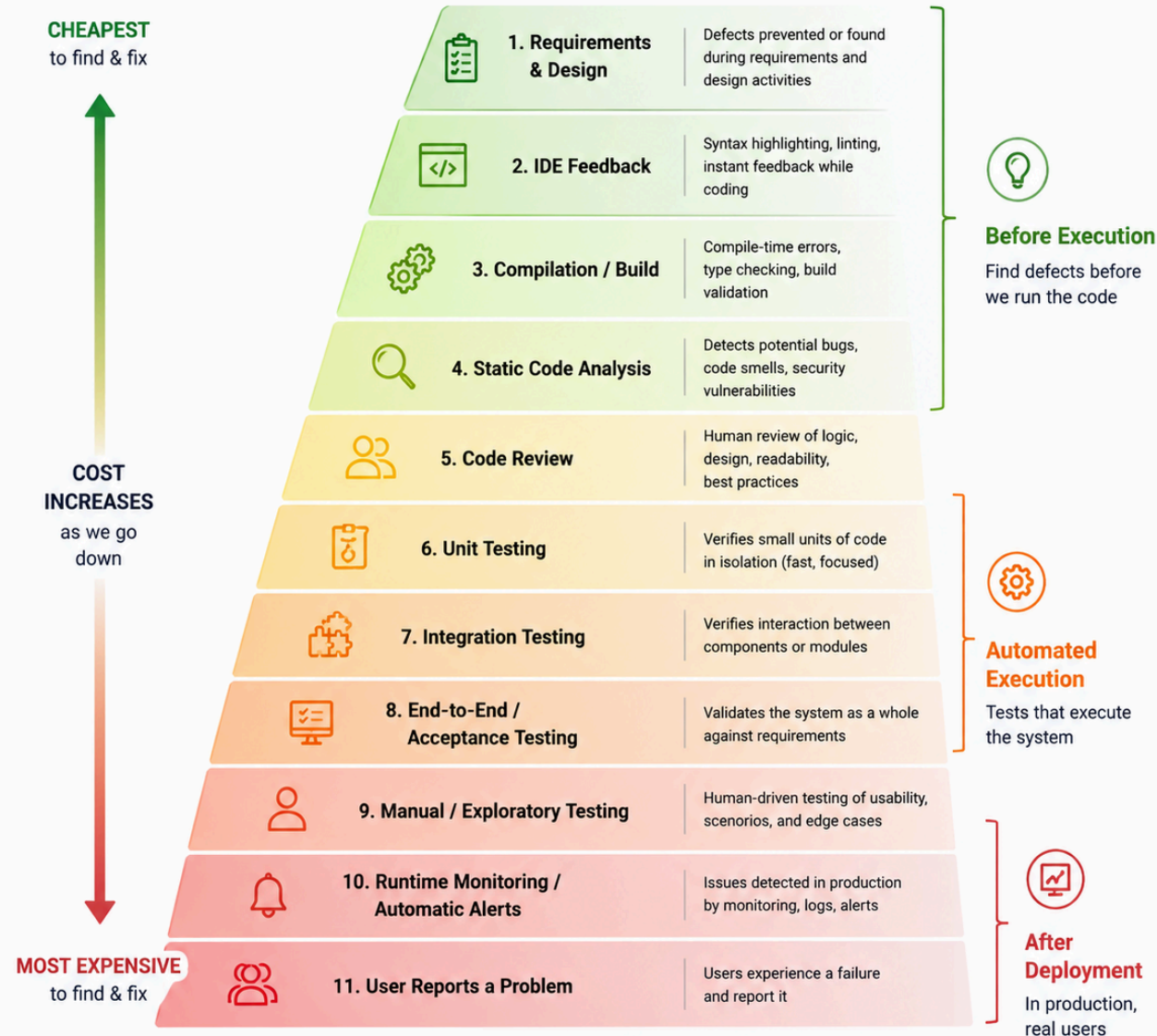
El *formatter* corre solo, y la discusión **deja de existir**:
el código se formatea de una manera porque
así está configurado el proyecto.

2 • Verificar sin ejecutar

El compilador y el análisis estático

Cost of Detecting a Software Defect

The later a defect is found, the more expensive it is to fix



Good engineering moves defect detection as far up the pyramid as possible.
Find early. Fix early. Save time, money and protect users.

Renombrás un campo

```
data class User(val fullName: String) // antes: name
```

```
println(user.name)
```

```
//           ^ Unresolved reference: name
```

```
class User:
```

```
    def __init__(self, full_name): ... # antes: name
```

```
print(user.name)
```

```
# AttributeError – pero recién cuando esa línea se ejecute
```

El compilador ya es un analizador estático

Es la herramienta de verificación que **más usan** y la que **menos piensan** como tal.

Revisa el 100% del código, en cada *build*, sin que nadie escriba una sola prueba.

¿Hasta dónde puede llegar?

```
enum Estado { Pendiente, Pagado, Cancelado } // agrego Cancelado

match estado {
  Estado::Pendiente => cobrar(),
  Estado::Pagado    => enviar(),
}
```

```
error[E0004]: non-exhaustive patterns: `Cancelado` not covered
```

Segunda barrera: análisis estático

¿Lo ves?

```
if (user.getRole() == "admin") {  
    grantAdminAccess(user);  
}
```

Compila. Pasa la revisión. Y es **siempre falso**.

La herramienta lo ve en un segundo

```
error: [ReferenceEquality] Comparison using reference  
equality instead of value equality  
    if (user.getRole() == "admin") {  
        ^
```

Nadie escribió un test para esto.
Nadie lo revisó. Y aun así apareció.

Seguir el dato hasta donde hace daño

```
String id = request.getParameter("id");  
// ...  
stmt.executeQuery("SELECT * FROM users WHERE id = " + id);
```

La herramienta marca `id` como **no confiable** y lo sigue por todo el programa hasta ver dónde termina.

El valor, en una línea

Un **test** encuentra errores donde alguien pensó en **buscarlos**.

Compilador y análisis estático los encuentran en **todo el código**, incluso donde nadie miró nunca.

Presencia y ausencia

Un test que pasa dice "**este caso anda**".
Nunca dice "no hay errores".

*Probar un programa puede mostrar la **presencia** de errores,
nunca su **ausencia**.*

— Edsger Dijkstra, 1970

El compilador sí puede decir "**ninguna línea hace esto**".

Y lo que ninguno de los dos ve

Ningún analizador te va a decir que la **fórmula del descuento está mal**.

Ningún test va a recorrer el camino que **nadie imaginó**.

No son redundantes: son **ortogonales**.

Cuando la herramienta se equivoca

```
if (config.isValid()) {  
    process(config.getName().trim());  
    //                ^ warning: possible null dereference  
}
```

`isValid()` ya garantiza que el nombre existe.
La herramienta **no tiene cómo saberlo**.

Sólo sirve si le creés

De cada diez avisos, ¿cuántos pueden estar mal antes de que el equipo **deje de leerlos**?

Depende de **cuánto te interrumpe**:

- El **compilador** te frena el *build* → casi **ninguno**
- El **linter en el PR** lo podés ignorar → hasta **1 de cada 10**

¿Y la IA?

```
// Devuelve el total con IVA incluido  
BigDecimal getTotal() {  
    return subtotal;  
}
```

El *linter* conoce **reglas**. El LLM lee **intención**.

Lo que ninguna regla nombró

- El **nombre** de la función no dice lo que hace
- El **comentario** contradice al código
- En los otros doce *endpoints* se valida el permiso — **en éste no**
- Estás sumando **pesos con dólares**: los dos son `BigDecimal`
- Un *copy-paste* donde **quedó una variable sin renombrar**

Nadie escribió una regla para nada de esto.

No es un compilador. Es un revisor.

La regla te da garantía sin criterio:
siempre encuentra lo mismo, y nunca inventa.

El LLM te da criterio sin garantía:
puede ver lo que nadie nombró — y también inventarlo.

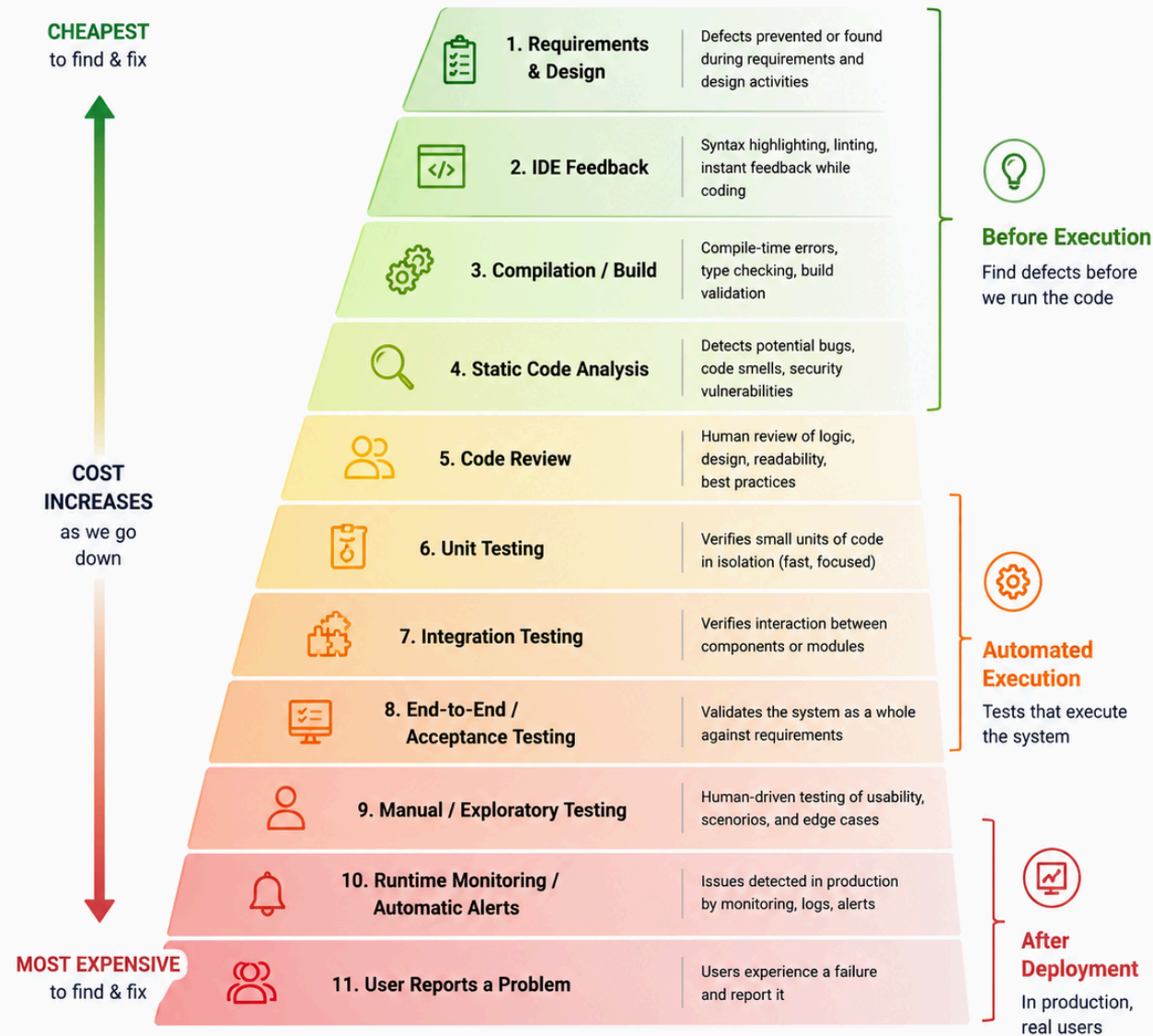
Por eso lo que funciona es ponerlos **en serie**:
el LLM propone, la regla verifica.

3 • Verificar ejecutando

Tests automáticos

Cost of Detecting a Software Defect

The later a defect is found, the more expensive it is to fix



Good engineering moves defect detection as far up the pyramid as possible.
Find early. Fix early. Save time, money and protect users.

¿Para qué testeamos?

No para demostrar que el código anda hoy.
Para poder **cambiarlo mañana** sin miedo.

Cada test es una **foto del comportamiento** en el momento
en que alguien lo consideró correcto.

Cuando falla no dice "está mal".

Dice "**cambiaste algo que alguien decidió que era así**".

Unitario vs integración

Unitario — una pieza sola, sin base de datos, sin red.

Milisegundos. Falla señalando exactamente qué se rompió.

Integración — varias piezas juntas, con la infraestructura real.

Segundos o minutos. Falla señalando que algo del conjunto no encaja.

El mismo sistema, dos tests

```
// unitario – la regla de negocio, sola  
var total = calcularTotal(items, DESCUENTO_10);  
assertEquals(90, total);
```

```
// integración – la pieza contra la base real  
repository.save(new Order(items));  
var recuperada = repository.findById(id);  
assertEquals(items, recuperada.getItems());
```

Testeable no es un accidente

Si algo es **difícil de testear**, casi siempre es porque está **mal diseñado**.

Acoplamiento alto, estado mutable compartido, dependencias escondidas: las mismas cosas que hacen difícil un test hacen difícil un cambio.

Coverage

Mide qué líneas se **ejecutaron** durante los tests.

No mide si **verificaste** algo.

```
@Test
void test() {
    calcularTotal(items, DESCUENTO_10);    // 100% de cobertura
}                                           // 0 assertions
```

4 • Construir siempre igual

Build y reproducibilidad

"En mi máquina anda"
no es una excusa:
es un bug del proyecto

Qué automatiza un *build system*

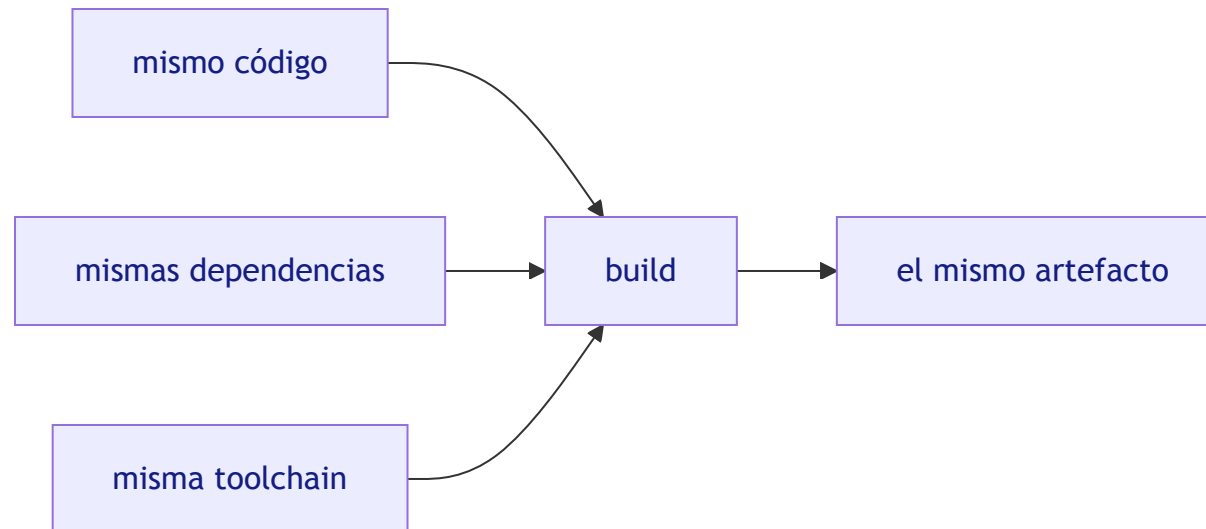
Una sola orden que siempre hace lo mismo:



Sin pasos manuales, sin orden que recordar,
sin "ah, antes tenés que correr esto otro".

Reproducibilidad

Las **mismas entradas** deben producir el **mismo artefacto**.



En cualquier máquina, en cualquier momento.

Dependencias

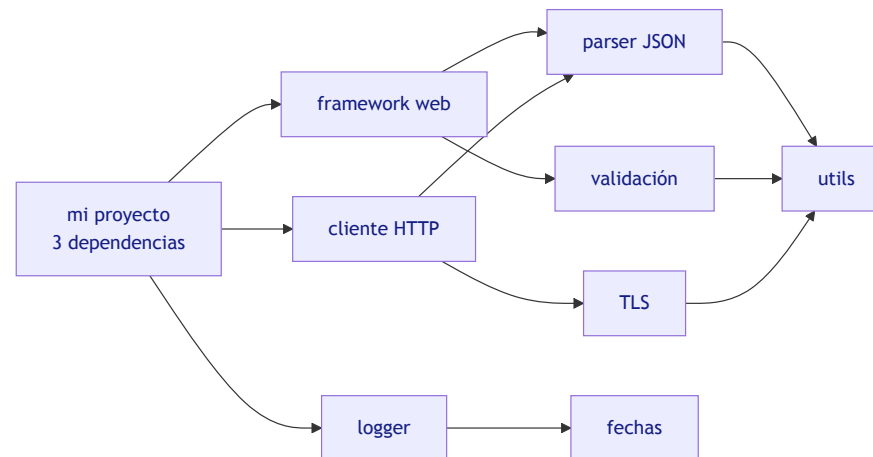
El código propio es la parte chica.

La mayor parte de lo que corre en producción **lo escribió otro**.

- Versiones **fijadas**, no "la última"
- **Lock files** — la versión exacta de todo el árbol
- Actualizar **a propósito**, no por accidente

Supply chain

Cada dependencia es código de terceros
que corre con **los mismos permisos** que el tuyo.



Y cada una trae **las suyas**.

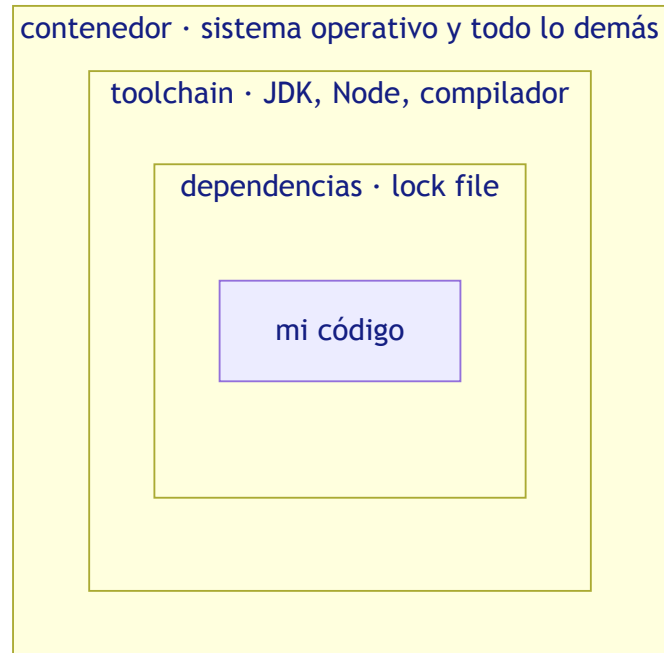
Un paréntesis: cuando la dependencia es el ataque

left-pad (2016) — se despublicó un paquete de once líneas y se rompieron miles de *builds* en cadena.

event-stream (2018) — el mantenedor cedió el proyecto; el nuevo dueño le agregó código para robar billeteras.

xz / liblzma (2024) — dos años ganándose la confianza del proyecto para introducir una puerta trasera en SSH.

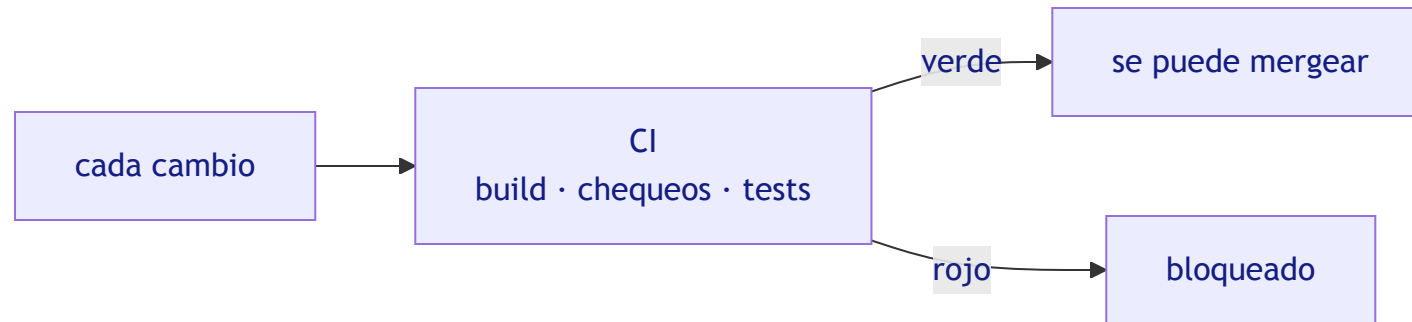
Fijar el ambiente entero



Cada capa se declara **en el proyecto**, no en la máquina de cada uno.

Y todo esto, corriendo solo

Un servidor construye, chequea y testea
en cada cambio, sin que nadie se acuerde.



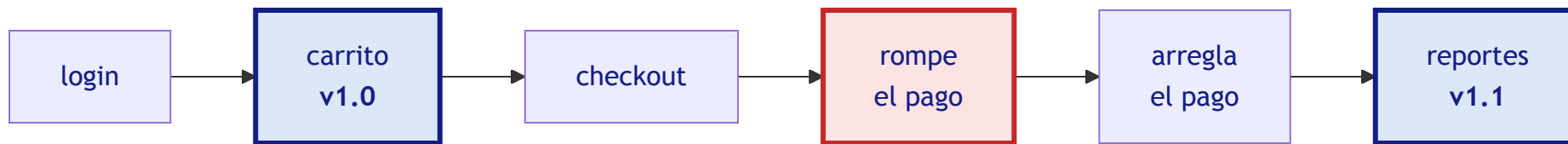
Eso es **Continuous Integration**.

Corridas reales — *formato-base* en rojo, *formato-cambio* en verde

5 • Registrar el cambio

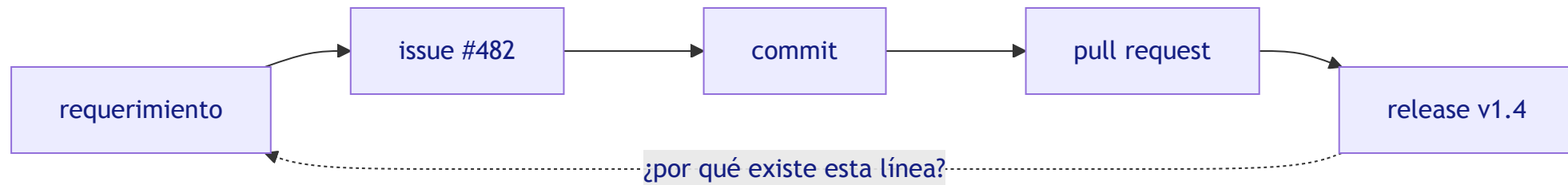
Versionado y trazabilidad

Qué resuelve un repositorio



Historial, integridad, *back-up* — y sobre todo
el **qué** / **cuándo** / **quién** / **por qué** de cada cambio.

Trazabilidad



La cadena se recorre en los **dos sentidos**.

Cadena real: issue #4 → PR #2 → release

6 • Revisar entre humanos

Code reviews y pair programming

¿Para qué revisamos código?

Antes de mergear, **otra persona lee el cambio**.
Eso es un **code review**.

- Encontrar **defectos**
- **Transferir conocimiento** — que no haya una sola persona que entienda cada parte
- **Responsabilidad compartida** sobre lo que se mergea
- Encontrar **mejores soluciones**

Qué se mira

- **Propósito** — ¿resuelve el problema que dice resolver?
- **Implementación** — ¿es la forma razonable de hacerlo?
- **Legibilidad** — ¿lo va a entender alguien dentro de seis meses?
- **Seguridad** — ¿abre algún agujero?
- **Tests** — ¿hay algo que verifique esto?

Por eso automatizamos todo lo automatizable

Si el *formatter* y el *linter* ya revisaron el estilo,
la persona puede revisar el **diseño**.

La atención humana es el recurso caro.
No gastarla en lo que una herramienta hace mejor.

Cómo dar *feedback*

"Esto está mal, no entendés cómo funciona el cache."

*"Si dos requests llegan juntos, ¿los dos escriben en el cache?
Me parece que puede haber una condición de carrera."*

[Review real sobre el PR #2](#)

Pair y mob programming

Dos personas, un teclado.

Driver — escribe.

Navigator — piensa un paso más adelante.

La revisión deja de ser un momento posterior:
pasa a ocurrir **mientras se escribe**.

Son la misma idea

Una **segunda cabeza** sobre el código.

Pairing — sincrónico. Sin espera, contexto compartido, más caro por hora.

PR — asincrónico. Barato por hora, con latencia y menos contexto.

No hay que elegir uno: hay que elegir **cuándo** cada uno.

7 • Transmitir el conocimiento

Documentación

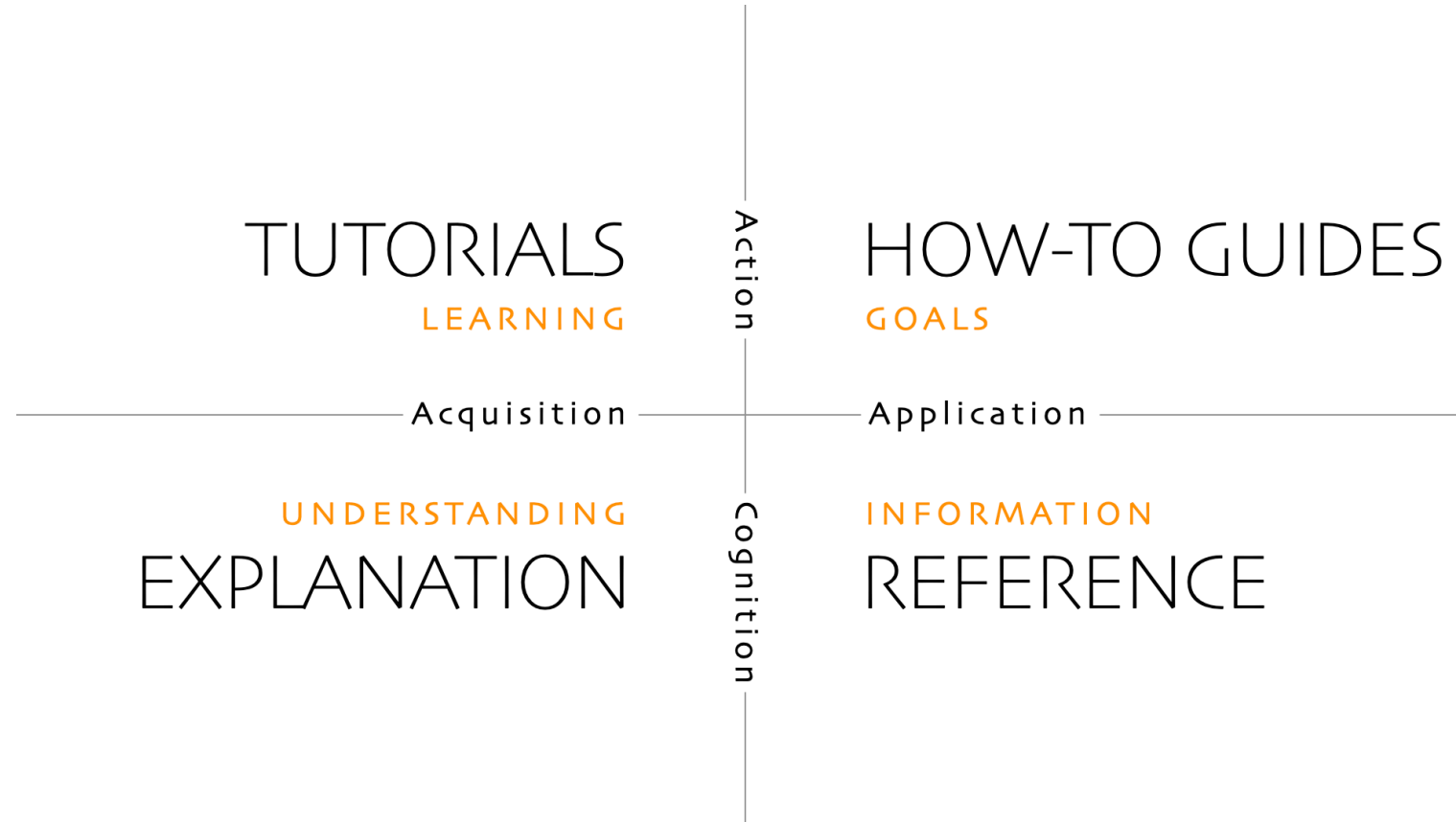
Tres audiencias distintas



De usuario — para quien usa el sistema, en sus distintos roles

Técnica interna — requerimientos, arquitectura, guías

Del proceso — cómo trabaja el equipo: flujos, convenciones, acuerdos



Cuatro documentos distintos, según **qué está haciendo** quien lee.

La documentación que sobrevive

La que está **cerca del código** y se actualiza con él.

- **Generada** desde el código o los tipos
- **Ejecutable** — ejemplos que corren en CI y fallan si mienten
- **Versionada** en el mismo repositorio

¿Y la IA?

Un modelo **reconstruye** la *reference* leyendo el código.

El **por qué** no lo puede inferir — no está ahí.

Pero sí puede ayudarte a **registrarlo**: preguntarte, ordenar la discusión, redactar el ADR mientras la decisión está fresca.

Lo que no está en el código, alguien lo tiene que **decir**.

Escribirlo dejó de ser la parte cara.

8 • Construir tus propias herramientas

Cuando la herramienta no existe

Hacerle la vida más fácil al equipo
es **parte del trabajo**, no una distracción.

Un script de veinte líneas que ahorra diez minutos por día
se paga en una semana.

Moldable development

Adaptar el **entorno** al dominio del problema,
en vez de adaptarse uno al entorno.

Herramientas hechas a medida del sistema que estás construyendo.

Tudor Gîrba — moldabledevelopment.com · [Glamorous Toolkit](#)

Otras formas de lo mismo

- **Notebooks** — [Jupyter](#), [clerk.vision](#)
- **Storybook** — desarrollar componentes de UI aislados
- **Live coding** — Smalltalk, Lisp: cambiar el programa mientras corre

Todas acortan el ciclo entre **hacer un cambio** y **ver qué pasó**.

¿Y la IA?

Construir una herramienta propia costaba una semana.
Ahora puede costar **una tarde**.

Y si es **chica y autocontenida**, mantenerla
también se volvió barato.

Lo que no se abarata: la herramienta que **nadie adoptó**.

El costo



Toda herramienta propia es **también código**:
alguien la tiene que entender y hacerse cargo.

Vale la pena cuando el problema es **específico del equipo**.
Si ya existe algo estándar, usar lo estándar.

Contra la entropía

La otra mitad del hilo

La máquina se acuerda de lo **verificable**.
Del resto se tiene que acordar el **equipo**.

- **Arreglar los problemas ni bien aparecen** — la ventana rota
- **Preguntarse por qué** — no quedarse en el síntoma
- **Retrospectivas** — mejorar el proceso, no sólo el código

El proceso también se degrada

Un proceso que nadie revisa acumula entropía igual que el código.

La **retrospectiva** es el mantenimiento del proceso.

Deuda técnica

Deuda es tomar una decisión subóptima **a conciencia**, para entregar antes, sabiendo que se va a pagar.

Lo que se hizo mal sin saberlo no es deuda.
Es simplemente **código malo**.

Ninguna de estas prácticas es gratis

Todas cuestan **tiempo hoy**
para ahorrar **tiempo mañana**.

La pregunta nunca es "¿es una buena práctica?".
Es "**¿el proyecto va a vivir lo suficiente para que se pague?**"

El mapa, completo

1. **Convenciones** — y el *formatter* que las hace cumplir
2. **Análisis estático** — verificar sin ejecutar
3. **Tests** — verificar ejecutando
4. **Build** — construir siempre igual
5. **Versionado** — el qué, cuándo, quién y por qué
6. **Reviews y pairing** — la segunda cabeza
7. **Documentación** — que no se desactualiza
8. **Herramientas propias** — cuando no existe la que necesitás

Gracias